

CLOUD COMPUTING

LAB NO # 10



SUBMITTED TO:

Sir Shohaib

SUBMITTED BY:

Name: Amina Noor

Reg No. 2023-BSE-007

Semester: V-A

CLOUD COMPUTING

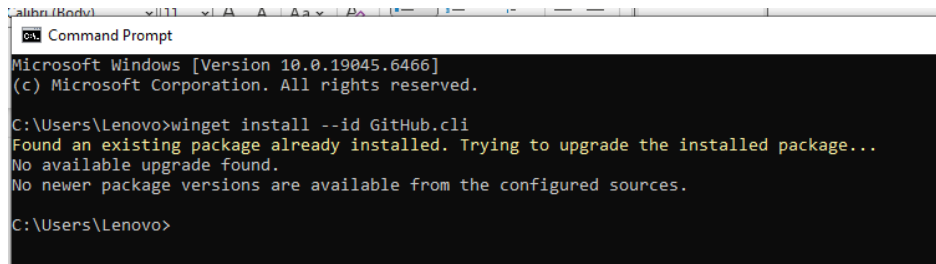
LAB NO # 10

Task 1: GitHub CLI + Codespaces Setup

Step 1: Install GitHub CLI (LOCAL MACHINE – Windows)

Open **Windows Terminal / PowerShell** (not Codespace).

Run: `winget install --id GitHub.cli`



```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

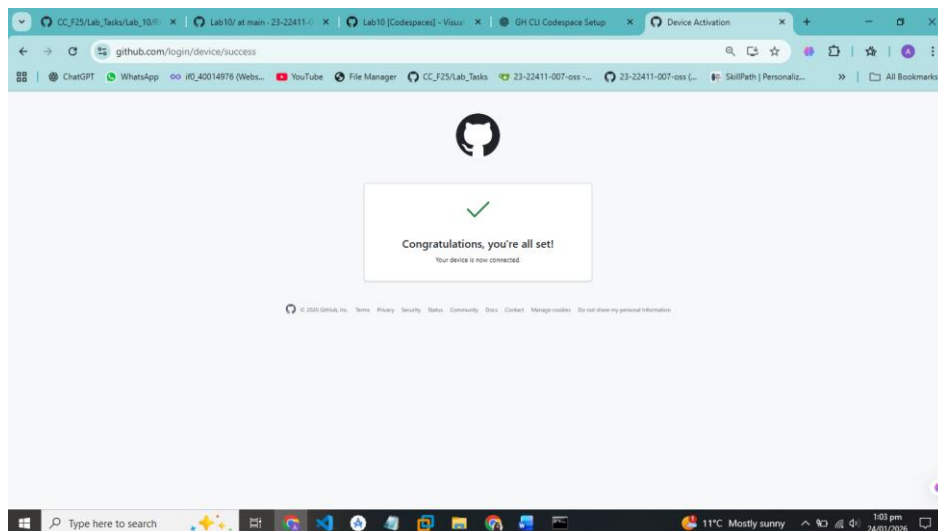
C:\Users\Lenovo>winget install --id GitHub.cli
Found an existing package already installed. Trying to upgrade the installed package...
No available upgrade found.
No newer package versions are available from the configured sources.

C:\Users\Lenovo>
```

Step 2: Authenticate GitHub CLI for Codespaces (LOCAL MACHINE)

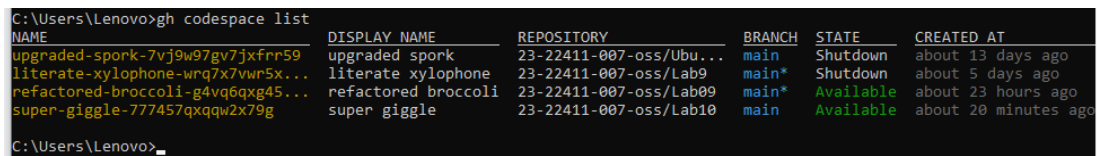
Still on your **local machine terminal**, run:

`gh auth login -s codespace`



Step 3: List Available Codespaces (LOCAL MACHINE)

Run: `gh codespace list`



```
C:\Users\Lenovo>gh codespace list
NAME                                DISPLAY_NAME    REPOSITORY                                BRANCH  STATE    CREATED AT
upgraded-spork-7vj9w97gv7jxfnr59  upgraded spork  23-22411-007-oss/Ubu...  main    Shutdown about 13 days ago
literate-xylophone-wrq7x7vwr5x...  literate xylophone  23-22411-007-oss/Lab9   main*   Shutdown about 5 days ago
refactored-broccoli-g4vq6qxg45...  refactored broccoli  23-22411-007-oss/Lab09  main*   Available about 23 hours ago
super-giggle-777457qxqgw2x79g      super giggle      23-22411-007-oss/Lab10  main    Available about 20 minutes ago

C:\Users\Lenovo>
```

Step 4: Connect to Codespace via SSH

Use the codespace name from previous step:

gh codespace ssh -c < super-giggle-777457qxqqw2x79g>

```
C:\Users\Lenovo>gh codespace ssh -c super-giggle-777457qxqqw2x79g
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@23-22411-007-oss  /workspaces/Lab10 (main) $
```

Task 2: AWS CLI + Terraform + Provider Setup

Install & Configure AWS CLI

Step A1: Install AWS CLI (skip ONLY if already installed)

Run **one command at a time**:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

```
unzip awscliv2.zip
```

```
sudo ./aws/install
```

Verify: `aws --version`

```
inflating: aws/dist/awscli/customizations/wizard/wizards/lambda/new-function
inflating: aws/dist/awscli/customizations/wizard/wizards/configure/_main.y
inflating: aws/dist/awscli/customizations/wizard/wizards/iam/new-role.yml
inflating: aws/dist/awscli/customizations/wizard/wizards/dynamodb/new-table
inflating: aws/dist/awscli/customizations/wizard/wizards/events/new-rule.y
inflating: aws/dist/awscli/data/ac.index
inflating: aws/dist/awscli/data/cli.json
inflating: aws/dist/awscli/data/metadata.json
  creating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/top_level.txt
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/WHEEL
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/INSTALLER
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/METADATA
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/RECORD
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/AUTHORS.rst
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/LICENSE
inflating: aws/dist/wheel-0.45.1.dist-info/METADATA
inflating: aws/dist/wheel-0.45.1.dist-info/WHEEL
inflating: aws/dist/wheel-0.45.1.dist-info/LICENSE.txt
inflating: aws/dist/wheel-0.45.1.dist-info/entry_points.txt
inflating: aws/dist/wheel-0.45.1.dist-info/REQUESTED
inflating: aws/dist/wheel-0.45.1.dist-info/RECORD
inflating: aws/dist/wheel-0.45.1.dist-info/direct_url.json
inflating: aws/dist/wheel-0.45.1.dist-info/INSTALLER
You can now run: /usr/local/bin/aws --version
@23-22411-007-oss →/workspaces/Lab10 (main) $ aws --version
aws-cli/2.33.6 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Step A2: Configure AWS CLI

Run: `aws configure`

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ aws configure
AWS Access Key ID [None]: AKIATFX3NIVRKUQ5GZFW
AWS Secret Access Key [None]: KT9ss07wDITe3ImecaC8QVwAHsvYqiXQhFuoQ0u
Default region name [None]: me-central-1
Default output format [None]: json
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Step A3: Show AWS config files

Run:

cat ~/.aws/credentials

cat ~/.aws/config

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ cat ~/.aws/credentials
cat ~/.aws/config
[default]
aws_access_key_id = AKIATFX3NIVRKUQ5GZFW
aws_secret_access_key = KT9ss07wDITe3ImecaC8QVwAHsvYqiXQhFuoQ0u
[default]
region = me-central-1
output = json
```

Step A4: Verify AWS Connectivity

Run: aws sts get-caller-identity

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ aws sts get-caller-identity
{
  "UserId": "AIDATFX3NIVRBVDJUFHNI",
  "Account": "218496845154",
  "Arn": "arn:aws:iam::218496845154:user/lab10"
}
@23-22411-007-oss →/workspaces/Lab10 (main) $
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Install Terraform CLI

Step B1: Add HashiCorp Repository

Run **exactly**:

wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [arch=\$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com \$(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update

sudo apt install terraform

Verify: which terraform

terraform --version

```

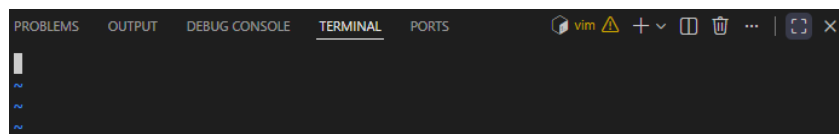
@23-22411-007-oss →/workspaces/Lab10 (main) $ wget -O - https://apt.releases.hashicorp.com/gpg
  sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
  echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release
  || lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list ...
id: EXPKEYSIG 23E7166788B63E1E Yarn Packaging <yarn@dan.cx>
E: The repository 'https://dl.yarnpkg.com/debian stable InRelease' is not signed.
N: Updating from such a repository can't be done securely, and is therefore disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration details.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  terraform
0 upgraded, 1 newly installed, 0 to remove and 76 not upgraded.
Need to get 30.6 MB of archives.
After this operation, 101 MB of additional disk space will be used.
Get:1 https://apt.releases.hashicorp.com noble/main amd64 terraform amd64 1.14.3-1 [30.6 MB]
Fetched 30.6 MB in 0s (176 MB/s)
Selecting previously unselected package terraform.
(Reading database ... 58629 files and directories currently installed.)
Preparing to unpack .../terraform_1.14.3-1_amd64.deb ...
Unpacking terraform (1.14.3-1) ...
Setting up terraform (1.14.3-1) ...
@23-22411-007-oss →/workspaces/Lab10 (main) $ which terraform
/usr/bin/terraform
Terraform v1.14.3
on linux_amd64
@23-22411-007-oss →/workspaces/Lab10 (main) $

```

Provider Configuration (Terraform)

Step C1: Create main.tf using Vim

Run: vim main.tf



Step C2: Write Provider Block

Press i (insert mode)

Paste **exactly this**:

```

provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

```

```

provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

```

Step C3: Save and Exit Vim

```

@23-22411-007-oss →/workspaces/Lab10 (main) $ vim main.tf
@23-22411-007-oss →/workspaces/Lab10 (main) $

```

Step C4: Initialize Terraform

Run: terraform init

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Step C5: Inspect Terraform Files

Show lock file:

cat .terraform.lock.hcl

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ cat .terraform.lock.hcl
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version = "6.28.0"
  hashes = [
    "h1:wzZdGs0FFmNqIgPyo9tKnGKJ37BGNSgwRrEXayL29+0=",
    "zh:0ba0d5eb6e0c6a933eb2befe3cdbf22b58fbc0337bf138f95bf0e8bb6e6df93e",
    "zh:23eacdd4e6db32cf0ff2ce189461bdbb62e46513978d33c5de4decc4670870ec",
    "zh:307b06a15fc00a8e6fd243abde2cbe5112e9d40371542665b91bec1018dd6e3c",
    "zh:37a02d5b45a9d050b9642c9e2e268297254192280df72f6e46641daca52e40ec",
    "zh:3da866639f07d92e734557d673092719c33ede80f4276c835bf7f231a669aa33",
    "zh:480060b0ba310d0f6b6a14d60b276698cb103c48fd2f7e2802ae47c963995ec6",
    "zh:57796453455c20db80d9168edbf125bf6180e1aae869de1546a2be58e4e405ec",
    "zh:69139cba772d4df8de87598d8d8a2b1b4b254866db046c061dccc79edb14e6b9",
    "zh:7312763259b859ff911c5452ca8bdf7d0be6231c5ea0de2df8f09d51770900ac",
    "zh:8d2d6f4015d3c155d7eb53e36f019a729aefb46ebfe13f3a637327d3a1402ecc",
    "zh:94ce589275c77308e6253f607de96919b840c2dd36c44aa798f693c9dd81af42",
    "zh:9b12af85486a96aedd8d7984b0ff811a4b42e3d88dad1a3fb4c0b580d04fa425",
    "zh:adaceec6a1bf4f5df1e12bd72cf52b72087c72efed078aef636f8988325b1a8b",
    "zh:d37be1ce187d94fd9df7b13a717c219964cd835c946243f096c6b230cdfd7e92",
    "zh:fe6205b5ca2ff36e68395cb8d3ae10a3728f405cdbcd46b206a515e1ebcf17a1",
  ]
}
```

Show terraform directory: ls .terraform/

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ ls .terraform/
providers
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Task 3: VPC & Subnet Creation + Verification

Step 1: Edit main.tf and Add Resources

Open the file:

vim main.tf

Press **i** (insert mode) and **append below your provider block**:

```
resource "aws_vpc" "development_vpc" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev_subnet_1" {
  vpc_id      = aws_vpc.development_vpc.id
  cidr_block   = "10.0.10.0/24"
  availability_zone = "me-central-1a"
}
```

```
provider "aws" {
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "development_vpc" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev_subnet_1" {
  vpc_id      = aws_vpc.development_vpc.id
  cidr_block   = "10.0.10.0/24"
  availability_zone = "me-central-1a"
}
```

Step 2: Save and Exit Vim

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ vim main.tf
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Step 3: Apply Terraform Configuration

Run: terraform apply

```

+ dhcp_options_id           = (known after apply)
+ enable_dns_hostnames      = (known after apply)
+ enable_dns_support        = true
+ enable_network_address_usage_metrics = (known after apply)
+ id                        = (known after apply)
+ instance_tenancy          = "default"
+ ipv6_association_id       = (known after apply)
+ ipv6_cidr_block           = (known after apply)
+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id       = (known after apply)
+ owner_id                  = (known after apply)
+ region                    = "me-central-1"
+ tags_all                  = (known after apply)
}

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_vpc.development_vpc: Creating...
aws_vpc.development_vpc: Creation complete after 1s [id=vpc-0fbfd0d9ba80ec19d]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 1s [id=subnet-04ed0115800235084]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
@23-22411-007-oss →/workspaces/Lab10 (main) $

```

Step 4: Get VPC ID and Subnet ID (IMPORTANT)

Run:

terraform state list

```

@23-22411-007-oss →/workspaces/Lab10 (main) $ terraform state list
aws_subnet.dev_subnet_1
aws_vpc.development_vpc
@23-22411-007-oss →/workspaces/Lab10 (main) $

```

Now get **Subnet ID**:

terraform state show aws_subnet.dev_subnet_1 (id= subnet-04ed0115800235084)

```

@23-22411-007-oss →/workspaces/Lab10 (main) $ terraform state show aws_subnet.dev_subnet_1
# aws_subnet.dev_subnet_1:
resource "aws_subnet" "dev_subnet_1" {
  arn                                = "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-04ed0115800235084"
  assign_ipv6_address_on_creation    = false
  availability_zone                  = "me-central-1a"
  availability_zone_id               = "mec1-az1"
  cidr_block                        = "10.0.10.0/24"
  customer_owned_ipv4_pool           = null
  enable_dns64                      = false
  enable_lni_at_device_index        = 0
  enable_resource_name_dns_a_record_on_launch = false
  enable_resource_name_dns_aaaa_record_on_launch = false
  id                                = "subnet-04ed0115800235084"
  ipv6_cidr_block                   = null
  ipv6_cidr_block_association_id     = null
  ipv6_native                        = false
  map_customer_owned_ip_on_launch   = false
  map_public_ip_on_launch           = false
  outpost_arn                       = null
  owner_id                          = "218496845154"
  private_dns_hostname_type_on_launch = "ip-name"
  region                            = "me-central-1"
  tags_all                          = {}
  vpc_id                            = "vpc-0fbfd0d9ba80ec19d"
}
@23-22411-007-oss →/workspaces/Lab10 (main) $

```


Get VPC ID:

terraform state show aws_vpc.development_vpc

id= vpc-0fbfd0d9ba80ec19d

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ terraform state show aws_vpc.development_vpc
# aws_vpc.development_vpc:
resource "aws_vpc" "development_vpc" {
  arn                    = "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-0fbfd0d9ba80ec19d"
  assign_generated_ipv6_cidr_block = false
  cidr_block             = "10.0.0.0/16"
  default_network_acl_id = "acl-028591b2d4c894bca"
  default_route_table_id = "rtb-0678435d3affe3366"
  default_security_group_id = "sg-039758b5fa6ff6e9f"
  dhcp_options_id        = "dopt-06f10cde7c4fc77fc"
  enable_dns_hostnames    = false
  enable_dns_support      = true
  enable_network_address_usage_metrics = false
  id                     = "vpc-0fbfd0d9ba80ec19d"
  instance_tenancy       = "default"
  ipv6_association_id     = null
  ipv6_cidr_block         = null
  ipv6_cidr_block_network_border_group = null
  ipv6_ipam_pool_id       = null
  ipv6_netmask_length     = 0
  main_route_table_id     = "rtb-0678435d3affe3366"
  owner_id                = "218496845154"
  region                 = "me-central-1"
  tags_all                = {}
}
```

Step 5: Verify Subnet Using AWS CLI

aws ec2 describe-subnets \

--filter "Name=subnet-id,Values= subnet-04ed0115800235084"

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ aws ec2 describe-subnets \
--filter "Name=subnet-id,Values= subnet-04ed0115800235084"
{
  "Subnets": [
    {
      "AvailabilityZoneId": "mec1-az1",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "218496845154",
      "AssignIpv6AddressOnCreation": false,
      "Ipv6CidrBlockAssociationSet": [],
      "SubnetArn": "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-04ed0115800235084",
      "EnableDns64": false,
      "Ipv6Native": false,
      "PrivateDnsNameOptionsOnLaunch": {
        "HostnameType": "ip-name",
        "EnableResourceNameDnsARecord": false,
        "EnableResourceNameDnsAAAARecord": false
      },
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "SubnetId": "subnet-04ed0115800235084",
      "State": "available",
      "VpcId": "vpc-0fbfd0d9ba80ec19d",
      "CidrBlock": "10.0.10.0/24",
      "AvailableIpAddressCount": 251,
      "AvailabilityZone": "me-central-1a",
      "DefaultForAz": false,
      "MapPublicIpOnLaunch": false
    }
  ]
}
```

Step 6: Verify VPC Using AWS CLI

aws ec2 describe-vpcs \

--filter "Name=vpc-id,Values= vpc-0fbfd0d9ba80ec19d"

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ aws ec2 describe-vpcs \
--filter "Name=vpc-id,Values= vpc-0fbfd0d9ba80ec19d"
{
  "Vpcs": [
    {
      "OwnerId": "218496845154",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-0dd0e36c647e46b0d",
          "CidrBlock": "10.0.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": false,
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-0fbfd0d9ba80ec19d",
      "State": "available",
      "CidrBlock": "10.0.0.0/16",
      "DhcpOptionsId": "dopt-06f10cde7c4fc77fc"
    }
  ]
}
```

Task 4: Data Source, Targeted Destroy & Tags

Part A — Data Source & Resource Creation

Step A1: Edit main.tf and Add Data Source + Resource

Open file:

vim main.tf

Press i and **add this BELOW** your existing resources:

```
data "aws_vpc" "existing_vpc" {
  default = true
}

resource "aws_subnet" "dev_subnet_1_existing" {
  vpc_id      = data.aws_vpc.existing_vpc.id
  cidr_block  = "172.31.48.0/24"
  availability_zone = "me-central-1a"
}
```

```

provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "development_vpc" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev_subnet_1" {
  vpc_id            = aws_vpc.development_vpc.id
  cidr_block        = "10.0.10.0/24"
  availability_zone = "me-central-1a"
}

data "aws_vpc" "existing_vpc" {
  default = true
}

resource "aws_subnet" "dev_subnet_1_existing" {
  vpc_id            = data.aws_vpc.existing_vpc.id
  cidr_block        = "172.31.48.0/24"
  availability_zone = "me-central-1a"
}

```

Step A2: Apply Configuration (Only New Subnet)

Run:

terraform apply

```

+ availability_zone            = me-central-1a
+ availability_zone_id        = (known after apply)
+ cidr_block                  = "172.31.48.0/24"
+ enable_dns64                = false
+ enable_resource_name_dns_a_record_on_launch = false
+ enable_resource_name_dns_aaaa_record_on_launch = false
+ id                          = (known after apply)
+ ipv6_cidr_block_association_id = (known after apply)
+ ipv6_native                 = false
+ map_public_ip_on_launch     = false
+ owner_id                    = (known after apply)
+ private_dns_hostname_type_on_launch = (known after apply)
+ region                      = "me-central-1"
+ tags_all                    = (known after apply)
+ vpc_id                      = "vpc-082ef0dc0c9f1d07d"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-0ee149e5a3947f029]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@23-22411-007-oss → /workspaces/Lab10 (main) $

```

Part B — Targeted Destroy, Refresh, Re-apply

Step B1: Targeted Destroy (ONLY ONE RESOURCE)

Run:

terraform destroy -target=aws_subnet.dev_subnet_1_existing

```

You are creating a plan with the -target option, which means that the result of this plan may not represent all of the changes by the current configuration.

The -target option is not for routine use, and is provided only for exceptional situations such as recovering from errors when Terraform specifically suggests to use it as part of an error message.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-0ee149e5a3947f029]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s

Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have been ignored. Output values may not be fully updated. Run the following command to verify that no other changes are pending:
    terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as recovering from errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.

Destroy complete! Resources: 1 destroyed.
bash: Type:: command not found
@23-22411-007-oss →/workspaces/Lab10 (main) $

```

Step B2: Refresh Terraform State

Run:

terraform refresh

```

@23-22411-007-oss →/workspaces/Lab10 (main) $ terraform refresh
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-0fbfd0d9ba80ec19d]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-082ef0dc0c9f1d07d]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-04ed0115800235084]
@23-22411-007-oss →/workspaces/Lab10 (main) $

```

Step B3: Re-Apply After Refresh

Run: terraform apply

```

+ availability_zone           = "me-central-1a"
+ availability_zone_id        = (known after apply)
+ cidr_block                  = "172.31.48.0/24"
+ enable_dns64                 = false
+ enable_resource_name_dns_a_record_on_launch = false
+ enable_resource_name_dns_aaaa_record_on_launch = false
+ id                          = (known after apply)
+ ipv6_cidr_block_association_id = (known after apply)
+ ipv6_native                  = false
+ map_public_ip_on_launch     = false
+ owner_id                    = (known after apply)
+ private_dns_hostname_type_on_launch = (known after apply)
+ region                      = "me-central-1"
+ tags_all                    = (known after apply)
+ vpc_id                      = "vpc-082ef0dc0c9f1d07d"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-054256d6e0b20b86a]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@23-22411-007-oss →/workspaces/Lab10 (main) $

```

Step B4: Destroy ALL Resources

Run: terraform destroy

```
- enable_dns_support           = true -> null
- enable_network_address_usage_metrics = false -> null
- id                           = "vpc-0fbfd0d9ba80ec19d" -> null
- instance_tenancy             = "default" -> null
- ipv6_netmask_length          = 0 -> null
- main_route_table_id          = "rtb-0678435d3affe3366" -> null
- owner_id                     = "218496845154" -> null
- region                       = "me-central-1" -> null
- tags                         = {} -> null
- tags_all                     = {} -> null
# (4 unchanged attributes hidden)
}
```

Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1: Destroying... [id=subnet-04ed0115800235084]
aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-054256d6e0b20b86a]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s
aws_subnet.dev_subnet_1: Destruction complete after 1s
aws_vpc.development_vpc: Destroying... [id=vpc-0fbfd0d9ba80ec19d]
aws_vpc.development_vpc: Destruction complete after 0s

Destroy complete! Resources: 3 destroyed.

@23-22411-007-oss →/workspaces/Lab10 (main) \$

Step B5: Terraform Plan (After Destroy)

Run:

terraform plan

```
# aws_vpc.development_vpc will be created
+ resource "aws_vpc" "development_vpc" {
+   arn                               = (known after apply)
+   cidr_block                        = "10.0.0.0/16"
+   default_network_acl_id           = (known after apply)
+   default_route_table_id           = (known after apply)
+   default_security_group_id        = (known after apply)
+   dhcp_options_id                  = (known after apply)
+   enable_dns_hostnames              = (known after apply)
+   enable_dns_support                = true
+   enable_network_address_usage_metrics = (known after apply)
+   id                               = (known after apply)
+   instance_tenancy                  = "default"
+   ipv6_association_id              = (known after apply)
+   ipv6_cidr_block                   = (known after apply)
+   ipv6_cidr_block_network_border_group = (known after apply)
+   main_route_table_id              = (known after apply)
+   owner_id                         = (known after apply)
+   region                           = "me-central-1"
+   tags_all                         = (known after apply)
}
```

Plan: 3 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

@23-22411-007-oss →/workspaces/Lab10 (main) \$

Step B6: Apply Again (Recreate Resources)

Run:

terraform apply

```
+ enable_dns_support           = true
+ enable_network_address_usage_metrics = (known after apply)
+ id                           = (known after apply)
+ instance_tenancy             = "default"
+ ipv6_association_id          = (known after apply)
+ ipv6_cidr_block               = (known after apply)
+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id          = (known after apply)
+ owner_id                     = (known after apply)
+ region                       = "me-central-1"
+ tags_all                     = (known after apply)
}
```

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.development_vpc: Creating...
aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-02e49e08feaa6f052]
aws_vpc.development_vpc: Creation complete after 1s [id=vpc-0e26ccd79393c483b]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 1s [id=subnet-0b2eddb2fb397f9b8]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

@23-22411-007-oss →/workspaces/Lab10 (main) \$

Part C — Tagging Resources

Step C1: Edit main.tf to Add Tags

Open file:

vim main.tf

Replace **ONLY the resource blocks** with this tagged version:

```
resource "aws_vpc" "development_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name     = "development"
    vpc_env = "dev"
  }
}

resource "aws_subnet" "dev_subnet_1" {
  vpc_id      = aws_vpc.development_vpc.id
  cidr_block  = "10.0.10.0/24"
  availability_zone = "me-central-1a"
```

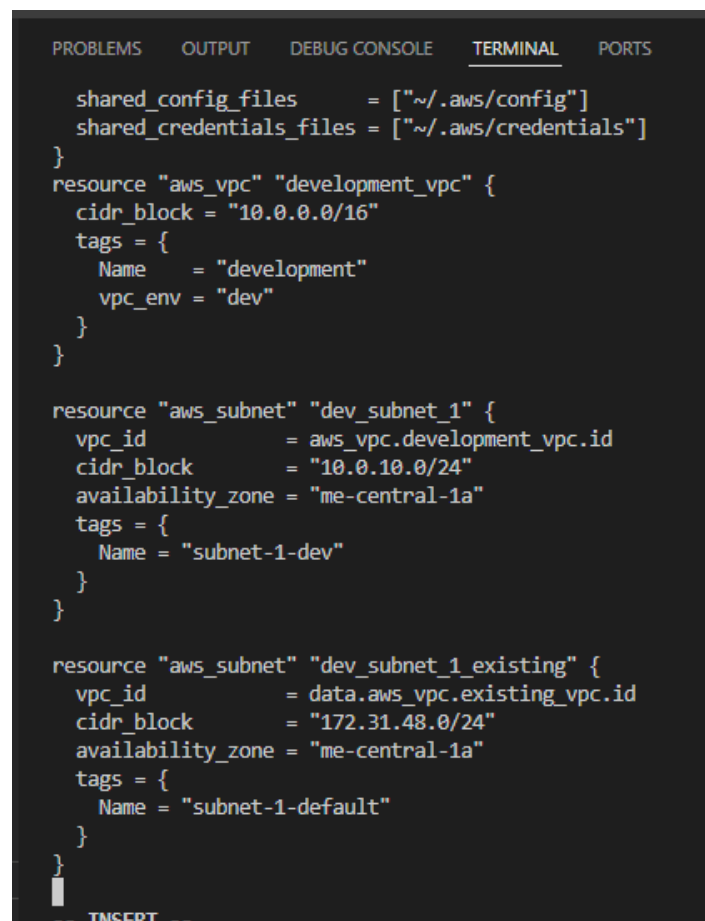
```

tags = {
  Name = "subnet-1-dev"
}
}

resource "aws_subnet" "dev_subnet_1_existing" {
  vpc_id      = data.aws_vpc.existing_vpc.id
  cidr_block   = "172.31.48.0/24"
  availability_zone = "me-central-1a"

  tags = {
    Name = "subnet-1-default"
  }
}

```



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

shared_config_files    = ["~/.aws/config"]
shared_credentials_files = ["~/.aws/credentials"]
}
resource "aws_vpc" "development_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name     = "development"
    vpc_env  = "dev"
  }
}

resource "aws_subnet" "dev_subnet_1" {
  vpc_id      = aws_vpc.development_vpc.id
  cidr_block   = "10.0.10.0/24"
  availability_zone = "me-central-1a"
  tags = {
    Name = "subnet-1-dev"
  }
}

resource "aws_subnet" "dev_subnet_1_existing" {
  vpc_id      = data.aws_vpc.existing_vpc.id
  cidr_block   = "172.31.48.0/24"
  availability_zone = "me-central-1a"
  tags = {
    Name = "subnet-1-default"
  }
}
-- INSERT --

```

Step C2: Apply Tagging Changes

Run:

terraform refresh

terraform apply -auto-approve

```
~ tags_all = {
  + "Name" = "subnet-1-default"
}
# (20 unchanged attributes hidden)
}

# aws_vpc.development_vpc will be updated in-place
~ resource "aws_vpc" "development_vpc" {
  id = "vpc-0e26ccd79393c483b"
  ~ tags = {
    + "Name" = "development"
    + "vpc_env" = "dev"
  }
  ~ tags_all = {
    + "Name" = "development"
    + "vpc_env" = "dev"
  }
  # (19 unchanged attributes hidden)
}

Plan: 0 to add, 3 to change, 0 to destroy.
aws_vpc.development_vpc: Modifying... [id=vpc-0e26ccd79393c483b]
aws_subnet.dev_subnet_1_existing: Modifying... [id=subnet-02e49e08feaa6f052]
aws_subnet.dev_subnet_1_existing: Modifications complete after 1s [id=subnet-02e49e08feaa6f052]
aws_vpc.development_vpc: Modifications complete after 1s [id=vpc-0e26ccd79393c483b]
aws_subnet.dev_subnet_1: Modifying... [id=subnet-0b2eddb2fb397f9b8]
aws_subnet.dev_subnet_1: Modifications complete after 1s [id=subnet-0b2eddb2fb397f9b8]

Apply complete! Resources: 0 added, 3 changed, 0 destroyed.
@23-22411-007-oss → /workspaces/Lab10 (main) $
```

Step C3: Remove One Tag (vpc_env)

Edit main.tf again: vim main.tf

```
resource "aws_vpc" "development_vpc" {

  cidr_block = "10.0.0.0/16"

  tags = {

    Name = "development"

  }

}
```

```
provider "aws" {
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "development_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "development"
  }
}

data "aws_vpc" "existing_vpc" {
  default = true
}

resource "aws_subnet" "dev_subnet_1" {
  vpc_id = aws_vpc.development_vpc.id
  cidr_block = "10.0.10.0/24"
  availability_zone = "me-central-1a"
  tags = {
    Name = "dev_subnet_1"
  }
}
```


Save & exit.

Step C4: Plan Tag Removal

Run: terraform plan

```
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-082ef0dc0c9f1d07d]
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-02e49e08feaa6f052]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-0b2eddb2fb397f9b8]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

# aws_vpc.development_vpc will be updated in-place
~ resource "aws_vpc" "development_vpc" {
  id          = "vpc-0e26ccd79393c483b"
  ~ tags      = {
    "Name"     = "development"
    - "vpc_env" = "dev" -> null
  }
  ~ tags_all  = {
    - "vpc_env" = "dev" -> null
      # (1 unchanged element hidden)
  }
  # (19 unchanged attributes hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform

Step C5: Apply Tag Removal

Run: terraform apply -auto-approve

```
aws_vpc.development_vpc: Refreshing state... [id=vpc-0e26ccd79393c483b]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-082ef0dc0c9f1d07d]
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-02e49e08feaa6f052]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-0b2eddb2fb397f9b8]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

# aws_vpc.development_vpc will be updated in-place
~ resource "aws_vpc" "development_vpc" {
  id          = "vpc-0e26ccd79393c483b"
  ~ tags      = {
    "Name"     = "development"
    - "vpc_env" = "dev" -> null
  }
  ~ tags_all  = {
    - "vpc_env" = "dev" -> null
      # (1 unchanged element hidden)
  }
  # (19 unchanged attributes hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.
aws_vpc.development_vpc: Modifying... [id=vpc-0e26ccd79393c483b]
aws_vpc.development_vpc: Modifications complete after 1s [id=vpc-0e26ccd79393c483b]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```

@23-22411-007-oss →/workspaces/Lab10 (main) \$

Task 5: State File Inspection & Terraform State Commands

Step 1: Destroy ALL Resources

Run:

terraform destroy

Type:

yes

```
- ipv6_netmask_length      = 0 -> null
- main_route_table_id     = "rtb-041a7128bec02ea12" -> null
- owner_id                = "218496845154" -> null
- region                  = "me-central-1" -> null
- tags                    = {
  - "Name" = "development"
} -> null
- tags_all                = {
  - "Name" = "development"
} -> null
# (4 unchanged attributes hidden)
}
```

Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-02e49e08feaa6f052]
aws_subnet.dev_subnet_1: Destroying... [id=subnet-0b2eddb2fb397f9b8]
aws_subnet.dev_subnet_1_existing: Destruction complete after 0s
aws_subnet.dev_subnet_1: Destruction complete after 0s
aws_vpc.development_vpc: Destroying... [id=vpc-0e26ccd79393c483b]
aws_vpc.development_vpc: Destruction complete after 1s

Destroy complete! Resources: 3 destroyed.

@23-22411-007-oss →/workspaces/Lab10 (main) \$

Step 2: Inspect terraform.tfstate (After Destroy)

Run:

cat terraform.tfstate

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 30,
  "lineage": "7e3f216d-6276-1ae4-a9c8-ddceecfed9a",
  "outputs": {},
  "resources": [],
  "check_results": null
}
```

@23-22411-007-oss →/workspaces/Lab10 (main) \$

- Must clearly show "resources": []

Step 3: Inspect Backup State File

Run:

cat terraform.tfstate.backup

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ cat terraform.tfstate.backup
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 25,
  "lineage": "7e3f216d-6276-1ae4-a9c8-ddceecfed9a",
  "outputs": {},
  "resources": [
    {
      "mode": "data",
      "type": "aws_vpc",
      "name": "existing_vpc",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "arn": "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-082ef0dc0c9f1d07d",
            "cidr_block": "172.31.0.0/16",
            "cidr_block_associations": [
              {
                "association_id": "vpc-cidr-assoc-0742c8946b197a094",
                "cidr_block": "172.31.0.0/16",
                "state": "associated"
              }
            ]
          },
          "default": true,
          "dhcp_options_id": "dopt-06f10cde7c4fc77fc",
          "enable_dns_hostnames": true,
          "enable_dns_support": true,

```

Step 4: Recreate Resources

Run:

terraform apply

Type:

yes

```
+ ipv6_cidr_block                = (known after apply)
+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id            = (known after apply)
+ owner_id                      = (known after apply)
+ region                        = "me-central-1"
+ tags                          = {
+   + "Name" = "development"
+ }
+ tags_all                      = {
+   + "Name" = "development"
+ }
}

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.development_vpc: Creating...
aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-0af6d2c84c7e96906]
aws_vpc.development_vpc: Creation complete after 2s [id=vpc-018cb4aa0e54918f7]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 0s [id=subnet-0aa19591c410c5431]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Step 5: Inspect State File (After Restore)

Run:

cat terraform.tfstate

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 34,
  "lineage": "7e3f216d-6276-1ae4-a9c8-ddceecfed9a",
  "outputs": {},
  "resources": [
    {
      "mode": "data",
      "type": "aws_vpc",
      "name": "existing_vpc",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "arn": "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-082ef0dc0c9f1d07d",
            "cidr_block": "172.31.0.0/16",
            "cidr_block_associations": [
              {
                "association_id": "vpc-cidr-assoc-0742c8946b197a094",
                "cidr_block": "172.31.0.0/16",
                "state": "associated"
              }
            ]
          },
          "default": true,
          "dhcp_options_id": "dopt-06f10cde7c4fc77fc",
          "enable_dns_hostnames": true
        }
      ]
    }
  ]
}
```

Step 6: Inspect Backup File (After Restore)

Run:

cat terraform.tfstate.backup

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ cat terraform.tfstate.backup
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 30,
  "lineage": "7e3f216d-6276-1ae4-a9c8-ddceecfed9a",
  "outputs": {},
  "resources": [],
  "check_results": null
}
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Step 7: List Managed Resources

Run:

terraform state list

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ terraform state list
data.aws_vpc.existing_vpc
aws_subnet.dev_subnet_1
aws_subnet.dev_subnet_1_existing
aws_vpc.development_vpc
```

Step 8: Show Full Attributes of One Resource

Choose **ONE** resource (recommended: VPC):

terraform state show aws_vpc.development_vpc

```
@23-22411-007-oss →/workspaces/Lab10 (main) $ terraform state show aws_vpc.development_vpc
# aws_vpc.development_vpc:
resource "aws_vpc" "development_vpc" {
  arn                                = "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-018cb4aa0e54918f7"
  assign_generated_ipv6_cidr_block  = false
  cidr_block                        = "10.0.0.0/16"
  default_network_acl_id           = "acl-04e2dbb2c3b4a1ad4"
  default_route_table_id           = "rtb-0c409e7a19381f3eb"
  default_security_group_id        = "sg-0c6357e59eca62003"
  dhcp_options_id                  = "dopt-06f10cde7c4fc77fc"
  enable_dns_hostnames              = false
  enable_dns_support                = true
  enable_network_address_usage_metrics = false
  id                               = "vpc-018cb4aa0e54918f7"
  instance_tenancy                  = "default"
  ipv6_association_id              = null
  ipv6_cidr_block                  = null
  ipv6_cidr_block_network_border_group = null
  ipv6_ipam_pool_id                = null
  ipv6_netmask_length              = 0
  main_route_table_id              = "rtb-0c409e7a19381f3eb"
  owner_id                         = "218496845154"
  region                           = "me-central-1"
  tags                             = {
    "Name" = "development"
  }
  tags_all                         = {
    "Name" = "development"
  }
}
```

Task 6: Terraform Outputs & Attributes Reporting

Part A — Basic Outputs (IDs & ARNs)

Step 1: Edit main.tf and Add Basic Outputs

Open file:

vim main.tf

Scroll to the **BOTTOM of the file** (very important — outputs always go at the end).

Press **i** and add:

```
output "dev-vpc-id" {
  value = aws_vpc.development_vpc.id
}

output "dev-subnet-id" {
  value = aws_subnet.dev_subnet_1.id
}

output "dev-vpc-arn" {
  value = aws_vpc.development_vpc.arn
}

output "dev-subnet-arn" {
```

```
value = aws_subnet.dev_subnet_1.arn
```

```
}
```

```
tags = {
  Name = "subnet-1-dev"
}
}

resource "aws_subnet" "dev_subnet_1_existing" {
  vpc_id          = data.aws_vpc.existing_vpc.id
  cidr_block      = "172.31.48.0/24"
  availability_zone = "me-central-1a"
  tags = {
    Name = "subnet-1-default"
  }
}

output "dev-vpc-id" {
  value = aws_vpc.development_vpc.id
}

output "dev-subnet-id" {
  value = aws_subnet.dev_subnet_1.id
}

output "dev-vpc-arn" {
  value = aws_vpc.development_vpc.arn
}

output "dev-subnet-arn" {
  value = aws_subnet.dev_subnet_1.arn
}
```

Step 2: Apply to Display Outputs

Run: terraform apply

```
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-018cb4aa0e54918f7]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-082ef0dc0c9f1d07d]
aws_subnet.dev_subnet_1_existing: Refreshing state... [id=subnet-0af6d2c84c7e96906]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-0aa19591c410c5431]

Changes to Outputs:
  + dev-subnet-arn = "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-0aa19591c410c5431"
  + dev-subnet-id  = "subnet-0aa19591c410c5431"
  + dev-vpc-arn    = "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-018cb4aa0e54918f7"
  + dev-vpc-id     = "vpc-018cb4aa0e54918f7"
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
dev-subnet-arn = "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-0aa19591c410c5431"
dev-subnet-id  = "subnet-0aa19591c410c5431"
dev-vpc-arn    = "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-018cb4aa0e54918f7"
dev-vpc-id     = "vpc-018cb4aa0e54918f7"
```

@23-22411-007-oss →/workspaces/Lab10 (main) \$

Part B — Expanded Outputs (Attributes)

Now we return **ALL required attributes**:

- cidr_block
- region / availability_zone
- tags.Name
- tags_all

Step 3: Expand Output Section in main.tf

Open file again:

vim main.tf

Go to the **output section** and **append** the following:

```
output "dev-vpc-cidr_block" {  
    value = aws_vpc.development_vpc.cidr_block  
}  
  
output "dev-vpc-region" {  
    value = aws_vpc.development_vpc.region  
}  
  
output "dev-vpc-tags_name" {  
    value = aws_vpc.development_vpc.tags["Name"]  
}  
  
output "dev-vpc-tags_all" {  
    value = aws_vpc.development_vpc.tags_all  
}  
  
output "dev-subnet-cidr_block" {  
    value = aws_subnet.dev_subnet_1.cidr_block  
}  
  
output "dev-subnet-region" {  
    value = aws_subnet.dev_subnet_1.availability_zone  
}  
  
output "dev-subnet-tags_name" {  
    value = aws_subnet.dev_subnet_1.tags["Name"]  
}  
  
output "dev-subnet-tags_all" {
```

```
value = aws_subnet.dev_subnet_1.tags_all
}
```

```
output "dev-vpc-region" {
  value = aws_vpc.development_vpc.region
}

output "dev-vpc-tags_name" {
  value = aws_vpc.development_vpc.tags["Name"]
}

output "dev-vpc-tags_all" {
  value = aws_vpc.development_vpc.tags_all
}

output "dev-subnet-cidr_block" {
  value = aws_subnet.dev_subnet_1.cidr_block
}

output "dev-subnet-region" {
  value = aws_subnet.dev_subnet_1.availability_zone
}

output "dev-subnet-tags_name" {
  value = aws_subnet.dev_subnet_1.tags["Name"]
}

output "dev-subnet-tags_all" {
  value = aws_subnet.dev_subnet_1.tags_all
}
}
-- INSERT --
```

Step 4: Apply and Record Expanded Outputs

Run: terraform apply

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
dev-subnet-arn = "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-0aa19591c410c5431"
dev-subnet-cidr_block = "10.0.10.0/24"
dev-subnet-id = "subnet-0aa19591c410c5431"
dev-subnet-region = "me-central-1a"
dev-subnet-tags_all = tomap({
  "Name" = "subnet-1-dev"
})
dev-subnet-tags_name = "subnet-1-dev"
dev-vpc-arn = "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-018cb4aa0e54918f7"
dev-vpc-cidr_block = "10.0.0.0/16"
dev-vpc-id = "vpc-018cb4aa0e54918f7"
dev-vpc-region = "me-central-1"
dev-vpc-tags_all = tomap({
  "Name" = "development"
})
dev-vpc-tags_name = "development"
@23-22411-007-oss → /workspaces/Lab10 (main) $
```

Cleanup: Delete Resources & State Verification

Step 1: Destroy ALL Terraform-Managed Resources

Run: terraform destroy

```
- dev-subnet-id      = "subnet-0aa19591c410c5431" -> null
- dev-subnet-region  = "me-central-1a" -> null
- dev-subnet-tags_all = {
  - Name = "subnet-1-dev"
} -> null
- dev-subnet-tags_name = "subnet-1-dev" -> null
- dev-vpc-arn         = "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-018cb4aa0e54918f7" -> null
- dev-vpc-cidr_block  = "10.0.0.0/16" -> null
- dev-vpc-id         = "vpc-018cb4aa0e54918f7" -> null
- dev-vpc-region      = "me-central-1" -> null
- dev-vpc-tags_all    = {
  - Name = "development"
} -> null
- dev-vpc-tags_name   = "development" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-0af6d2c84c7e96906]
aws_subnet.dev_subnet_1: Destroying... [id=subnet-0aa19591c410c5431]
aws_subnet.dev_subnet_1: Destruction complete after 1s
aws_vpc.development_vpc: Destroying... [id=vpc-018cb4aa0e54918f7]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s
aws_vpc.development_vpc: Destruction complete after 1s

Destroy complete! Resources: 3 destroyed.
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Step 2: Inspect State Files After Cleanup

View current state file: cat terraform.tfstate

View backup state file: cat terraform.tfstate.backup

```
● @23-22411-007-oss →/workspaces/Lab10 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 41,
  "lineage": "7e3f216d-6276-1ae4-a9c8-ddceecfed9a",
  "outputs": {},
  "resources": [],
  "check_results": null
}
● @23-22411-007-oss →/workspaces/Lab10 (main) $ cat terraform.tfstate.backup
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 36,
  "lineage": "7e3f216d-6276-1ae4-a9c8-ddceecfed9a",
  "outputs": {
    "dev-subnet-arn": {
      "value": "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-0aa19591c410c5431",
      "type": "string"
    },
    "dev-subnet-cidr_block": {
      "value": "10.0.10.0/24",
      "type": "string"
    },
    "dev-subnet-id": {
      "value": "subnet-0aa19591c410c5431",
      "type": "string"
    }
  }
}
```

Step 3: Re-Apply & Compare State (Verification Only)

This step proves you understand **state vs backup behavior**. You may or may not be required to submit screenshots — do it anyway for safety. Recreate resources: terraform apply

```
aws_subnet.dev_subnet_1_existing: Creating...
aws_vpc.development_vpc: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-0e0b9eda7fb58c66e]
aws_vpc.development_vpc: Creation complete after 2s [id=vpc-0bddec235aabbbb85]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 0s [id=subnet-0939b0ce5d952c93c]
```

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

```
dev-subnet-arn = "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-0939b0ce5d952c93c"
dev-subnet-cidr_block = "10.0.10.0/24"
dev-subnet-id = "subnet-0939b0ce5d952c93c"
dev-subnet-region = "me-central-1a"
dev-subnet-tags_all = tomap({
  "Name" = "subnet-1-dev"
})
dev-subnet-tags_name = "subnet-1-dev"
dev-vpc-arn = "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-0bddec235aabbbb85"
dev-vpc-cidr_block = "10.0.0.0/16"
dev-vpc-id = "vpc-0bddec235aabbbb85"
dev-vpc-region = "me-central-1"
dev-vpc-tags_all = tomap({
  "Name" = "development"
})
dev-vpc-tags_name = "development"
```

```
@23-22411-007-oss →/workspaces/Lab10 (main) $
```

Now compare again: cat terraform.tfstate

cat terraform.tfstate.backup

```
Plan: 0 to add, 0 to change, 3 to destroy.
```

Changes to Outputs:

```
- dev-subnet-arn      = "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-0939b0ce5d952c93c" -> null
- dev-subnet-cidr_block = "10.0.10.0/24" -> null
- dev-subnet-id        = "subnet-0939b0ce5d952c93c" -> null
- dev-subnet-region    = "me-central-1a" -> null
- dev-subnet-tags_all  = {
  - Name = "subnet-1-dev"
} -> null
- dev-subnet-tags_name = "subnet-1-dev" -> null
- dev-vpc-arn          = "arn:aws:ec2:me-central-1:218496845154:vpc/vpc-0bddec235aabbbb85" -> null
- dev-vpc-cidr_block   = "10.0.0.0/16" -> null
- dev-vpc-id           = "vpc-0bddec235aabbbb85" -> null
- dev-vpc-region       = "me-central-1" -> null
- dev-vpc-tags_all     = {
  - Name = "development"
} -> null
- dev-vpc-tags_name    = "development" -> null
aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-0e0b9eda7fb58c66e]
aws_subnet.dev_subnet_1: Destroying... [id=subnet-0939b0ce5d952c93c]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s
aws_subnet.dev_subnet_1: Destruction complete after 1s
aws_vpc.development_vpc: Destroying... [id=vpc-0bddec235aabbbb85]
aws_vpc.development_vpc: Destruction complete after 0s
```

Destroy complete! Resources: 3 destroyed.

```
@23-22411-007-oss →/workspaces/Lab10 (main) $
```